# Reinforcement Learning for Robotic Control under Sim-to-Sim Domain Shift

Samuel Clemos Matteo Marello Shaheer Abdullah Umesh Bansari
Politecnico di Torino
Group 68 – Reinforcement Learning

## Abstract

*Policies trained in simulation often fail to transfer when test-time dynamics differ from training conditions—a challenge that limits the practical deployment of RL for robotics. This work approaches the problem from two angles: algorithm design and domain transfer robustness. We implement REINFORCE, REINFORCE with a constant baseline, and Actor-Critic from scratch on the MuJoCo Hopper locomotion task, comparing their variance–bias trade-offs directly. We then study sim-to-sim transfer on PandaPush via Stable-Baselines3, using a source domain (1 kg cube) and a target domain (5 kg cube) to simulate a controlled reality gap. Unadapted source-to-target transfer serves as the lower bound; training on the target domain directly gives the upper bound. To close this gap, we implement Uniform Domain Randomisation (UDR) and Automatic Domain Randomisation (ADR) over cube mass. Among the Hopper algorithms, Actor-Critic performs best at 1000 episodes. For PandaPush, SAC outperforms PPO at 100k timesteps and is used throughout all 500k experiments. The main finding is that UDR raises source-to-target success from 0.50 to 1.00, matching the upper bound in success rate and achieving a comparable mean return, without ever training on target dynamics.*

## 1 Introduction

**Reinforcement learning fundamentals.** Reinforcement Learning (RL) is a framework in which an autonomous *agent* learns to behave by interacting with an *environment*. At each discrete time step, the agent observes the current *state* $s_t$, selects an *action* $a_t$ according to its *policy* $\pi_\theta$, and receives a scalar *reward* $r_t$ that evaluates how good that action was. The agent's goal is to find the policy that maximises the expected *return*—the discounted sum of future rewards [1]. The agent is never told what the correct action is; it discovers useful behaviour through exploration and the reward signal alone, which makes RL well suited to problems where the optimal strategy is difficult to specify by hand.

**Why RL for robotics?** Robotic control requires selecting sequences of motor commands that accomplish physical tasks under uncertainty. Classical model-based controllers need an accurate dynamics model, which is difficult to handcraft for contact-rich manipulation or locomotion. RL can discover controllers directly from experience, adapting to complex or partially unknown dynamics without an explicit model [2, 3]. RL has produced strong results across locomotion, grasping, assembly, and dexterous manipulation.

**The sim-to-real gap.** Real-robot experience is slow, expensive, and potentially unsafe. Random exploration can damage a robot or its surroundings before any useful behaviour emerges. For this reason, modern robot-RL pipelines *train in simulation* and then transfer the policy to the real world. However, simulators are imperfect models of reality: differences in mass, friction, contact geometry, sensor noise, and actuator delay create a *reality gap*. A policy that achieves near-perfect performance in simulation may fail catastrophically on the physical robot [4, 6]. This project reproduces that problem in a controlled *sim-to-sim* setting, where source and target are both simulations with a deliberately injected dynamics mismatch—a fivefold increase in cube mass.

**Lower bound, upper bound, and domain randomisation.** A principled evaluation framework requires two baselines. The *lower bound* is the source-trained policy evaluated on the target domain without adaptation: it exposes the full degradation caused by the mass shift. The *upper bound* is the policy trained and evaluated entirely in the target domain: it represents the best achievable performance since the policy has perfect access to the test-time dynamics. Domain randomisation methods aim to close the gap between these two extremes by training on a *distribution* of dynamics rather than a single set of parameters [5].

**Paper structure.** Section 2 reviews related work. Section 3 describes the algorithms and experimental setup. Section 4 reports results. Section 5 provides analysis and limitations.[1]

[1] Code repository: https://github.com/MatteoMarello/Reinforcement_Learning

## 2 Related Work

**Policy-gradient RL.** Policy-gradient algorithms directly optimise a parameterised policy $\pi_\theta$ by following the gradient of expected return [1]. REINFORCE uses Monte-Carlo episode returns: it is unbiased but high-variance. Adding a *baseline* that is independent of the action does not change the expected gradient but can substantially reduce its variance. Actor-Critic methods replace the Monte-Carlo return with a one-step temporal-difference advantage, using a learned *value function* (the critic) to evaluate states during the episode. This reduces variance at the cost of introducing a bias term from critic approximation error.

**Modern continuous-control RL.** Proximal Policy Optimisation (PPO) [9] is an on-policy algorithm that constrains policy updates via a clipped surrogate objective, providing stable learning across diverse tasks. Soft Actor-Critic (SAC) [10] is an off-policy maximum-entropy method that adds an entropy bonus to the objective, encouraging exploration and avoiding premature convergence. SAC reuses past transitions from a replay buffer, making it substantially more sample-efficient than on-policy methods at the same timestep budget. Both algorithms are available through Stable-Baselines3 [11]. MuJoCo [12] provides the physics engine; panda-gym [13] provides the PandaPush manipulation task.

**Domain randomisation.** Uniform Domain Randomisation (UDR) trains on a fixed distribution of simulator parameters, chosen to cover the deployment conditions [5, 6]. The policy becomes robust because the target conditions are never surprising at test time. Automatic Domain Randomisation (ADR) removes the need for manual range selection by adaptively expanding or contracting the parameter distribution boundaries based on recent agent performance—if the agent succeeds easily near a boundary, the boundary is pushed outward; if it fails, it contracts [8]. ADR was used to train the OpenAI Dactyl hand [8]; a comprehensive review is given by [7].

## 3 Methodology

### 3.1 RL formulation

We model each task as a Markov Decision Process with states $s_t$, actions $a_t$, rewards $r_t$, discount factor $\gamma$, and stochastic policy $\pi_\theta(a_t|s_t)$. The objective is

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right]. \tag{1}$$

For REINFORCE, the policy-gradient theorem gives

$$\nabla_\theta J(\theta) \approx \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\,(G_t - b), \tag{2}$$

where $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ is the return from step $t$ and $b$ is a constant baseline. For Actor-Critic, a critic $V_\phi(s_t)$ is trained alongside the actor; the one-step advantage is

$$A_t = r_t + \gamma V_\phi(s_{t+1})(1 - d_t) - V_\phi(s_t), \tag{3}$$

where $d_t$ is the terminal flag. The actor minimises $-\log \pi_\theta(a_t|s_t) A_t$ and the critic minimises the squared TD error.

### 3.2 Hopper policy-gradient implementation

Hopper-v4 is a continuous-control environment from MuJoCo [12] in which a one-legged planar robot must move forward without falling. The observation has 11 dimensions (joint angles, velocities, and position); actions are 3-dimensional bounded torques applied to the joints. We implement all three algorithms from scratch in PyTorch. The actor is a two-hidden-layer MLP with 64 units and $\tanh$ nonlinearities, outputting the mean and log-standard-deviation of a diagonal Gaussian policy. Actions are squashed through a $\tanh$ transformation to enforce bounds, with the corresponding change-of-variables correction included in the log-probability. For Actor-Critic, a separate linear value head estimates $V_\phi(s)$ from the shared trunk.

All Hopper experiments use seed 42, $\gamma = 0.99$, actor learning rate $3 \times 10^{-4}$, critic learning rate $10^{-3}$, and 64 hidden units. Evaluation is deterministic and runs every 100 episodes; we report the best and final evaluation returns. Short 50-episode smoke tests are excluded from the official comparison as they are too brief to measure convergence.

### 3.3 PandaPush: environments and evaluation protocol

PandaPush [13] is a goal-conditioned manipulation task in which a Franka Panda arm must push a cube to a target location. We define:

- **Source domain**: cube mass $m_s = 1\,\text{kg}$. The policy is trained here.
- **Target domain**: cube mass $m_t = 5\,\text{kg}$. The fivefold increase in inertia means the cube responds much more sluggishly to the same push force.

Three canonical transfer configurations define the evaluation protocol:

- **source→source** – trained and evaluated in the source domain. Sanity check that the algorithm can solve the task.
- **source→target** – trained in source, evaluated in target. This is the *lower bound*: it measures raw transfer performance without any adaptation.
- **target→target** – trained and evaluated in the target domain. This is the *upper bound*: the policy has full access to the test-time dynamics and represents the ceiling performance.

Phase 3 compares PPO and SAC at 100k timesteps; SAC is selected for the 500k final baselines based on better transfer performance. Phase 4 augments SAC source training with

Table 1. Hopper Phase 1/2 results. Deterministic evaluation return; 50-episode smoke tests excluded. **Bold**: best value per column.

| Method | Episodes | Baseline | Best | Final |
|---|---|---|---|---|
| REINFORCE | 1k | – | 346.7 | 243.2 |
| REINF+ baseline | 1k | 20 | 223.2 | 219.7 |
| REINF+ baseline | 1k | 1000 | 181.0 | 97.8 |
| Actor-Critic | 1k | – | **466.2** | **466.2** |
| REINF+ baseline | 15k | 20 | 444.3 | **321.3** |
| Actor-Critic | 15k | – | **1010.4** | 168.0 |

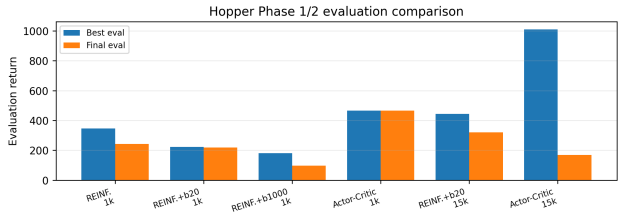



Figure 1. Hopper evaluation return for all methods. Actor-Critic reaches the highest values but its 15k run degrades at the end; REINFORCE with baseline 20 provides the most stable long-run result.

domain randomisation over cube mass. **UDR** samples a new mass uniformly from a broad fixed interval $[m_{\min}, m_{\max}]$ at every episode reset; the range is chosen to reflect plausible mass variation in the source domain without assuming knowledge of the exact target mass, forcing the agent to learn behaviour that is robust across a wide spectrum of dynamics. **ADR** starts from a narrow interval centred on $m_s$ and adaptively widens each boundary when the policy succeeds near it and contracts it when the policy fails, building a self-paced curriculum. All evaluations use dense reward, end-effector control, 50 deterministic test episodes, and evaluation seed 1234.

# 4 Experiments and Results

## 4.1 Hopper: policy-gradient comparison

Table 1 reports the Phase 1/2 comparison. At 1000 episodes, Actor-Critic achieves the best return (466.2), clearly ahead of both REINFORCE variants. The baseline value turns out to matter a lot: $b = 20$ is far better than $b = 1000$, because an excessively large baseline flips most advantages negative, effectively reversing the update direction for the majority of steps and destabilising training. In the 15 000-episode runs, Actor-Critic reaches the highest peak (1010.4) but degrades sharply by the end, finishing at 168.0. REINFORCE with $b = 20$ is more conservative on the peak but ends at 321.3, making it the more reliable choice over the long run.

Figure 1 shows the evaluation-return curves and Figure 2 shows the training curves for the 1000-episode runs.

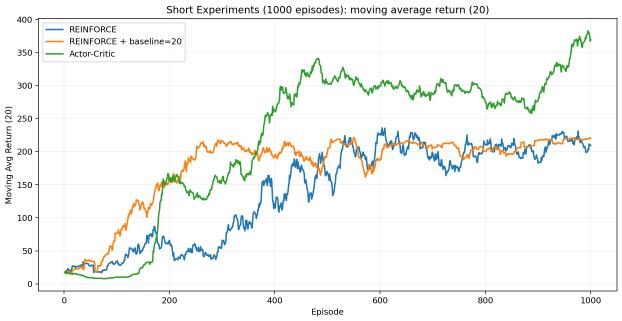



Figure 2. Hopper training curves for the 1000-episode runs. Actor-Critic converges faster and reaches higher values, reflecting the reduced variance from step-level critic feedback.

Table 2. PPO vs SAC on the source→targettransfer case at 100k timesteps (50 deterministic episodes). Higher return (less negative) and higher success rate are both better.

| Algorithm | Mean return | Success |
|---|---|---|
| PPO (source→target) | −3.77 | 0.12 |
| SAC (source→target) | −3.26 | **0.24** |

## 4.2 PandaPush: PPO vs SAC at 100k timesteps

Before the 500k experiments, we compare PPO and SAC at 100k timesteps to select the algorithm for domain randomisation. The comparison is summarised in Table 2 for the key source→targettransfer case. SAC achieves mean return −3.26 and success rate 0.24, while PPO achieves −3.77 and 0.12. Figure 3 shows the success rates across all configurations. SAC's advantage is consistent on the source→targetcase, which is the primary criterion; the most likely explanation is that its off-policy replay buffer lets it reuse past transitions more efficiently at the same timestep budget. We therefore use SAC for all subsequent 500k experiments and domain randomisation.

## 4.3 PandaPush: SAC 500k baselines

Table 3 reports the official SAC 500k results (training seed 0, evaluation seed 1234). The target→targetconfiguration reaches success rate 1.00, confirming that SAC can perfectly solve PandaPush when trained on the correct dynamics. Interestingly, the source→targetresult is slightly *better* than source→sourceon both metrics (−2.073 vs. −2.276 mean return; 0.58 vs. 0.52 success rate). Rather than interpreting this as evidence that the mass shift is beneficial, we attribute it to evaluation variability over 50 episodes: with such a small sample, random fluctuations in the episode outcomes can easily produce a difference of this magnitude. The gap that matters is between either lower-bound configuration (success $\leq 0.58$) and the target-domain upper bound (success 1.00), which motivates the domain randomisation experiments in Phase 4. Figure 4 provides a visual summary.

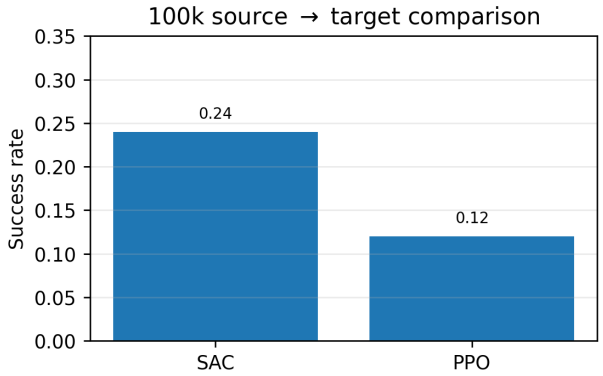



Figure 3. PandaPush success rates for PPO and SAC at 100k timesteps. SAC is consistently superior on the key source→targetcase, motivating its selection for the 500k runs.

Table 3. Official Phase 3 SAC 500k baselines (evaluation seed 1234, 50 episodes).

| Configuration | Mean return | Std | Success |
|---|---|---|---|
| source→source | −2.276 | 2.083 | 0.52 |
| source→target | −2.073 | 1.980 | 0.58 |
| target→target | −0.475 | 0.284 | 1.00 |

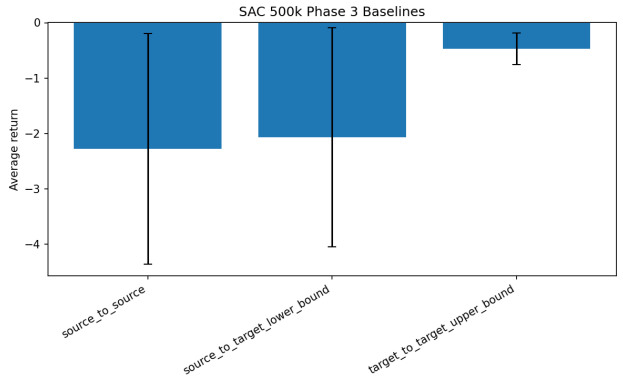



Figure 4. SAC 500k evaluation results across all three transfer configurations. The target-trained upper bound achieves perfect success; the source-trained lower bound shows the unadapted transfer gap.

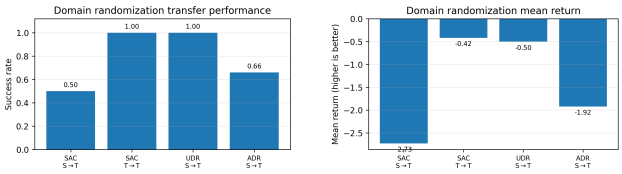
### 4.4 Domain randomisation: UDR vs ADR

Table 4 reports the Phase 4 results. For a fair comparison, the baseline used here was evaluated within the same experimental pipeline as UDR and ADR, rather than mixing results generated during different stages of the project. The minor difference between the two source→targetbaseline values (success 0.50 here vs. 0.58 in Table 3) is consistent with 50-episode evaluation variance over equivalent runs.

UDR is the strongest method. It raises

Table 4. Phase 4 domain randomisation results (500k SAC, 50 deterministic episodes).

| Configuration | Mean return | Std | Success |
|---|---|---|---|
| SAC source→target(baseline) | −2.727 | 2.328 | 0.50 |
| SAC target→target(upper bound) | −0.416 | 0.236 | 1.00 |
| UDR source→source | −0.491 | 0.549 | 0.98 |
| UDR source→target | −0.502 | 0.420 | 1.00 |
| ADR source→source | −1.883 | 1.795 | 0.68 |
| ADR source→target | −1.920 | 1.893 | 0.66 |




(a) Success rates. (b) Mean returns.

Figure 5. Phase 4 domain randomisation results. UDR fully closes the transfer gap in success rate and achieves a mean return close to the upper bound; ADR achieves partial improvement on both metrics.

source→targetsuccess from 0.50 to 1.00 and achieves mean return $-0.50$, very close to the upper-bound value of $-0.42$. The UDR source→sourceresult (success 0.98) confirms that the policy is also near-perfect in the source domain, demonstrating that training on a wide mass distribution does not hurt performance on the nominal source mass. ADR also improves over the baseline (0.50 → 0.66 for source→target), but remains substantially weaker than UDR. Notably, ADR source→target(0.66) is slightly lower than ADR source→source(0.68), suggesting that the curriculum did not adequately cover the target mass region during the available training budget.

## 5 Discussion

**UDR versus ADR.** The key to UDR's success is that it uses a *broad* mass range rather than a narrow distribution centred on the target mass. The range was chosen to reflect plausible variation around the source domain without assuming any prior knowledge of the exact target value. Because the target mass falls comfortably within this range, the agent encounters it routinely during training and does not treat it as an out-of-distribution case at evaluation time. ADR, by contrast, must *earn* coverage of the target region through a self-paced curriculum. If the success threshold is set too conservatively, or the training budget runs out before the boundaries reach $m_t$, the policy simply never encounters the hardest transfer conditions. Our results are consistent with this failure mode: ADR source→sourceand source→targetboth end up around 0.67, nearly indistinguishable, suggesting the curriculum had not yet expanded far enough to make the target mass

genuinely challenging. A more thorough ADR study would tune the boundary expansion rate and success threshold, or run multiple seeds to separate algorithm behaviour from single-run variance.

**Actor-Critic instability on Hopper.** The 15k Actor-Critic run (peak 1010.4, final 168.0) is a good illustration of a known failure mode. Early in training, the critic provides step-level advantage estimates that are low-variance and speed up the actor's progress considerably. The problem is that as the policy improves and visits new states, the critic's coverage lags behind. If bootstrapping error accumulates in these poorly-covered regions, the advantage estimates become unreliable, and a single bad update can destroy a policy that took thousands of episodes to build. REINFORCE with baseline 20, despite being noisier per update, avoids this because its gradient always comes from the actual return rather than a learned approximation. The $b = 20$ vs $b = 1000$ comparison also confirms a known practical rule: the baseline should be close to the expected return, not arbitrarily large—$b = 1000$ sits far above early returns, which causes most advantages to be negative and effectively inverts the learning signal for the first several hundred episodes.

**Limitations.** The most significant constraint is the use of a single training seed throughout (seed 42 for Hopper, seed 0 for PandaPush). With one run per configuration it is impossible to know how much of the observed differences are genuine algorithmic effects versus luck of initialisation. Proper statistical comparisons would require multiple seeds and reported standard errors. Beyond that, the entire study is sim-to-sim: the "target" is still a simulator, and a physical robot would introduce additional sources of error—sensor noise, motor backlash, control latency—that are absent here. Finally, domain randomisation was applied only to cube mass. In a real transfer scenario one would likely need to randomise friction, contact geometry, and actuator gains as well; restricting to a single parameter limits the generality of the conclusions.

## 6 Conclusion

We implemented three policy-gradient algorithms from scratch on Hopper and benchmarked PPO and SAC on PandaPush using Stable-Baselines3. Actor-Critic was the best-performing Hopper method at 1000 episodes. On PandaPush, SAC was consistently stronger than PPO at 100k timesteps and was used for all subsequent 500k experiments. The domain randomisation results are the most notable outcome of the project: UDR raised source-to-target success from 0.50 to 1.00, matching the target upper bound in success rate and achieving a mean return close to the upper-bound return while never training on fixed target dynamics; ADR improved over the unadapted baseline ($0.50 \rightarrow 0.66$) but did not match UDR in our run. Overall, these findings confirm that training on a distribution of dynamics, even over a single physical parameter, can be enough to close a substantial transfer gap. Future work should extend randomisation to multiple parameters, evaluate with multiple seeds, and validate on physical hardware.

## 7 Acknowledgments

The authors declare that the scientific content of the project report, including the experimental methodology, result selection, discussion, and conclusions, was produced and validated by the group members. Generative AI tools were used only for limited language-level support, such as spelling, grammar, formatting, and consistency checks, under human supervision.

For the development of the submitted project code, the group used Claude Opus 4.7, published by Anthropic, and ChatGPT 5.5 Thinking, published by OpenAI. These tools were used under continuous human supervision for implementation support, debugging assistance, code-structure review, formatting checks, and clarification of programming issues. They were not used to autonomously decide the experimental methodology, select official results, tune hyperparameters without human validation, or draw the scientific conclusions. All algorithmic choices, experimental configurations, result selection, and final interpretations were reviewed and validated by the authors, who take full responsibility for the submitted code and report.

## References

[1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[2] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *Int. J. Robotics Research*, 32(11):1238–1274, 2013.

[3] P. Kormushev, S. Calinon, and D. G. Caldwell. Reinforcement learning in robotics: Applications and real-world challenges. *Robotics*, 2(3):122–148, 2013.

[4] S. Höfer et al. Perspectives on sim2real transfer for robotics. *IEEE RA-L*, 6(3):4480–4487, 2021.

[5] J. Tobin et al. Domain randomization for transferring deep neural networks from simulation to the real world. In *IROS*, 2017.

[6] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *ICRA*, 2018.

[7] F. Muratore, C. Eilers, M. Gienger, and J. Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9, 2022.

[8] I. Akkaya et al. Solving Rubik's Cube with a robot hand. *arXiv:1910.07113*, 2019.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[10] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft Actor-Critic: Off-policy maximum entropy deep RL with a stochastic actor. In *ICML*, 2018.

[11] A. Raffin et al. Stable-Baselines3: Reliable RL implementations. *JMLR*, 22(268):1–8, 2021.

[12] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IROS*, 2012.

[13] Q. Gallouédec, N. Cazin, E. Dellandréa, and L. Chen. panda-gym: Open-source goal-conditioned environments for robotic learning, 2021.